

PREDICTIVE MODELING AND INVESTMENT STRATEGY OPTIMIZATION USING THE SMARKET DATASET

Khush Domadiya

Chenghui Tan (Sabrina)

Mohanasundaram Murugesan

Amisha Farhana Shaik

INTRODUCTION AND OBJECTIVES

- Smarket dataset - contains a 1250 days of S&P 500 index percentage returns.

- Year: year of the return
- Lag1 - Lag5: 1 to 5 days lagged returns
- Volume: the number of shares traded on the previous day (in billions)
- Today: the percentage return on the day in question
- Direction: whether the return was positive or negative on the day in question

- Objectives:
 - Calculate the return on investments
 - Create and analyze regression models to predict the percentage returns
 - Analyze and answer few questions at the end

APPROACH

- Split the dataset into **training (2004 and earlier)** and **test (2005)** sets.

```
train_data = Smarket[Smarket['Year'] < 2005]
test_data = Smarket[Smarket['Year'] == 2005]

# Display the number of rows in each dataset
print("Training data shape:", train_data.shape)
print("Test data shape:", test_data.shape)

# Optionally, display the first few rows of each dataset
print("\nTraining Data (2004 and earlier):\n", train_data.head())
print("\nTest Data (2005):\n", test_data.head())
```

Training data shape: (998, 9)
Test data shape: (252, 9)

Training Data (2004 and earlier):

	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Volume	Today	Direction
0	2001	0.381	-0.192	-2.624	-1.055	5.010	1.1913	0.959	Up
1	2001	0.959	0.381	-0.192	-2.624	-1.055	1.2965	1.032	Up
2	2001	1.032	0.959	0.381	-0.192	-2.624	1.4112	-0.623	Down
3	2001	-0.623	1.032	0.959	0.381	-0.192	1.2760	0.614	Up
4	2001	0.614	-0.623	1.032	0.959	0.381	1.2057	0.213	Up

Test Data (2005):

	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Volume	Today	Direction
998	2005	-0.134	0.008	-0.007	0.715	-0.431	0.7869	-0.812	Down
999	2005	-0.812	-0.134	0.008	-0.007	0.715	1.5108	-1.167	Down
1000	2005	-1.167	-0.812	-0.134	0.008	-0.007	1.7210	-0.363	Down
1001	2005	-0.363	-1.167	-0.812	-0.134	0.008	1.7389	0.351	Up
1002	2005	0.351	-0.363	-1.167	-0.812	-0.134	1.5691	-0.143	Down

CALCULATING TOTAL INVESTMENT RETURN OVER ENTIRE DATASET

```
def calculate_total_investment_return(data, initial_investment_amount):  
    """  
    Calculate the total return on an investment invested at the beginning of the dataset  
    and held until the end of the dataset.  
  
    Parameters:  
    - data (DataFrame): The dataset containing historical returns.  
    - initial_investment_amount (float): The amount of money initially invested.  
  
    Returns:  
    - float: The final value of the investment at the end of the dataset.  
    """  
    # Convert daily percentage returns to numerical format  
    daily_percentage_returns = data['Today'] / 100  
  
    # Compute cumulative return factor: (1 + daily_return) for each day  
    cumulative_return_factors = (1 + daily_percentage_returns).cumprod()  
    print(cumulative_return_factors)  
    # Calculate the final value of the investment based on the cumulative return factor  
    final_investment_value = initial_investment_amount * cumulative_return_factors.iloc[-1]  
    print(cumulative_return_factors.iloc[-1])  
  
    return final_investment_value  
  
# Example usage:  
initial_investment_amount = 1000 # Example: $1000  
  
# Calculate the total return on the investment over the entire dataset  
final_investment_value = calculate_total_investment_return(Smarket, initial_investment_amount)  
  
print(f"The final value of an initial investment of ${initial_investment_amount} is: ${final_investment_value:.2f}")
```

OUTPUT

```
0      1.009590
1      1.020009
2      1.013654
3      1.019878
4      1.022050
...
1245   0.975187
1246   0.965874
1247   0.967129
1248   0.964247
1249   0.959532
Name: Today, Length: 1250, dtype: float64
0.9595322561695054
The final value of an initial investment of $1000 is: $959.53
```

Interpretation:

The cumulative return over the dataset is **negative**, as the final cumulative return factor is below 1.0. This suggests that despite some positive days, the overall performance of the market in this period resulted in a net loss.

CALCULATING TOTAL INVESTMENT RETURN AT THE END OF A PARTICULAR DAY

```
def calculate_return_on_investment(data, investment_value, day_index):
    # Ensure the day_index is within the bounds of the dataset
    if day_index < 0 or day_index >= len(data):
        raise IndexError("Day index is out of range.")

    # Investment is made before the percentage return for that day is known
    # so we use the return for that day to compute the end value
    daily_return = data['Today'].iloc[day_index] / 100
    final_value = investment_value * (1 + daily_return)

    return final_value

# Example usage:
investment_value = 1000 # Initial investment amount

# Calculate the return for the first 10 days as an example
#for day_index in range(10):
day_index=10
day_index-=1
final_value = calculate_return_on_investment(Smarket, investment_value, day_index)
print(f"Day {day_index+1}: Initial Investment = ${investment_value}, Final Value = ${final_value:.2f}")

Day 10: Initial Investment = $1000, Final Value = $1002.87
```

LINEAR REGRESSION MODEL TO PREDICT THE PERCENTAGE RETURN.

- Dataset: S&P 500 daily returns data from 2001 to 2005
- Predictor variables : 'Lag1', 'Lag2', 'Lag3', 'Lag4', 'Lag5', 'Volume'
- Train Test Split : Training dataset of all days from 2004 and earlier. The test set is data from 2005
- Dependent variable : Today i.e the percentage return on that day

CODE FOR LINEAR REGRESSION

```
# Load the dataset
Smarket = load_data('Smarket')

# Filter the data
train_data = Smarket[Smarket['Year'] <= 2004]
test_data = Smarket[Smarket['Year'] == 2005]
# Features and target variable
features = ['Lag1', 'Lag2', 'Lag3', 'Lag4', 'Lag5', 'Volume']
X_train = train_data[features]
y_train = train_data['Today']
# Add constant to the training features
X_train = sm.add_constant(X_train)

# Initialize and fit the OLS model using statsmodels
ols_model = sm.OLS(y_train, X_train).fit()

# Prepare the test data
X_test = test_data[features] # Select features for the test set
y_test = test_data['Today'] # Actual values for comparison

# Add constant to the test features
X_test = sm.add_constant(X_test)

# Predict the 'Today' variable using the OLS model
y_pred = ols_model.predict(X_test)

# Print predicted values and compare with actual
#print("Predicted values:", y_pred.values)
#print("Actual values:", y_test.values)

#summary
print(ols_model.summary())

# Calculate performance metrics
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print the performance metrics
print(f"Mean Squared Error: {mse:.2f}")
print(f"R-squared: {r2:.2f}")
```

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Create a dataframe with the features and constant
X_vif = sm.add_constant(X_train)

# Calculate VIF for each feature
vif_data = pd.DataFrame()
vif_data["Feature"] = X_vif.columns
vif_data["VIF"] = [variance_inflation_factor(X_vif.values, i) for i in range(X_vif.shape[1])]

print(vif_data)
```

LINEAR REGRESSION RESULTS

```

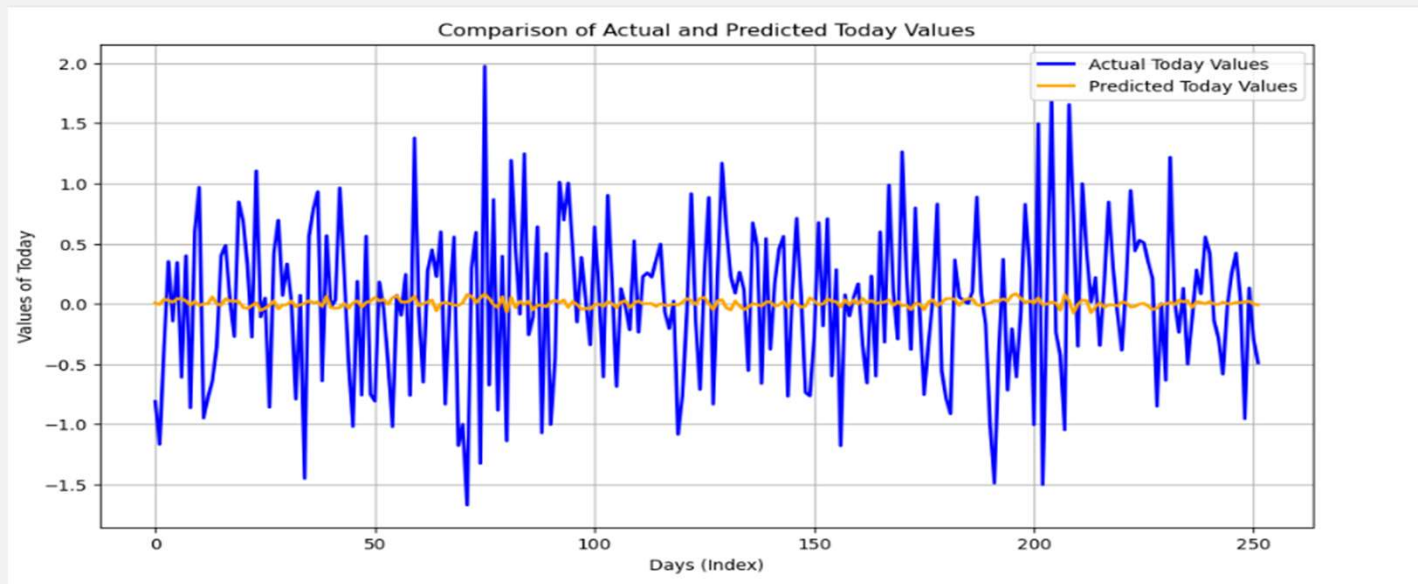
=====
                        OLS Regression Results
=====
Dep. Variable:          Today      R-squared:                0.002
Model:                  OLS        Adj. R-squared:            -0.004
Method:                 Least Squares      F-statistic:             0.3626
Date:                   Wed, 25 Sep 2024    Prob (F-statistic):       0.903
Time:                   10:57:34          Log-Likelihood:          -1620.8
No. Observations:       998              AIC:                    3256.
Df Residuals:           991              BIC:                    3290.
Df Model:                6
Covariance Type:        nonrobust
=====
                        coef    std err          t      P>|t|      [0.025    0.975]
-----
const                -0.0146      0.205     -0.071     0.943     -0.417     0.388
Lag1                  -0.0217      0.032     -0.684     0.494     -0.084     0.041
Lag2                  -0.0106      0.032     -0.332     0.740     -0.073     0.052
Lag3                  -0.0033      0.032     -0.104     0.917     -0.066     0.059
Lag4                  -0.0060      0.032     -0.188     0.851     -0.068     0.056
Lag5                  -0.0394      0.031     -1.254     0.210     -0.101     0.022
Volume                0.0110      0.147      0.075     0.940     -0.278     0.300
=====
Omnibus:                49.299    Durbin-Watson:           2.000
Prob(Omnibus):           0.000    Jarque-Bera (JB):        143.022
Skew:                    0.165    Prob(JB):                 8.77e-32
Kurtosis:                4.825    Cond. No.                  11.0
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
Mean Squared Error: 0.42
R-squared: 0.00

```

	Feature	VIF
0	const	27.656125
1	Lag1	1.003460
2	Lag2	1.005034
3	Lag3	1.006164
4	Lag4	1.008210
5	Lag5	1.002516
6	Volume	1.020027

COMPARISON OF ACTUAL AND PREDICTED TODAY VALUES



LOGSITIC REGRESSION MODEL TO PREDICT THE PERCENTAGE RETURN

Predicting Positive Returns on the S&P 500

- Key Points:
- Objective: Predict whether the return will be positive using logistic regression.
- Dataset: S&P 500 daily returns data from 2001 to 2005.
- Training set: Data until 2004; Test set: Data from 2005.

CODE FOR LOGSITIC REGRESSION

#QUESTION 4 - Create and analyze a logistic regression model to predict whether the return will be positive or negative.

```
import pandas as pd
from ISLP import load_data
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Load the dataset
Smarket = load_data('Smarket')
```

```
# Create binary target variable: 1 if 'Today' > 0 (positive return), 0 otherwise
Smarket['PositiveReturn'] = (Smarket['Today'] > 0).astype(int)
```

```
# Filter the data
train_data = Smarket[Smarket['Year'] <= 2004]
test_data = Smarket[Smarket['Year'] == 2005]
```

```
# Features and target variable
features = ['Lag1', 'Lag2', 'Lag3', 'Lag4', 'Lag5', 'Volume']
X_train = train_data[features]
y_train = train_data['PositiveReturn']
X_test = test_data[features]
y_test = test_data['PositiveReturn']
```

```
# Fit the Model
# Initialize and fit the Logistic regression model
logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(X_train, y_train)
```

```
# Predict probabilities on the test set
```

```
y_probs = logistic_model.predict_proba(X_test)[:, 1] # Get probabilities for the positive class
```

```
# Set a custom threshold
```

```
custom_threshold = 0.5
```

```
# Predict based on the custom threshold
```

```
y_pred_custom = (y_probs >= custom_threshold).astype(int)
```

```
# Performance metrics
```

```
accuracy = accuracy_score(y_test, y_pred_custom)
```

```
conf_matrix = confusion_matrix(y_test, y_pred_custom)
```

```
class_report = classification_report(y_test, y_pred_custom)
```

```
print(f"Accuracy: {accuracy:.2f}")
```

```
print("Confusion Matrix:")
```

```
print(conf_matrix)
```

```
print("Classification Report:")
```

```
print(class_report)
```

LOGISTIC REGRESSION RESULTS

Accuracy: 0.48

Confusion Matrix:

```
[[ 81  30]
 [101  40]]
```

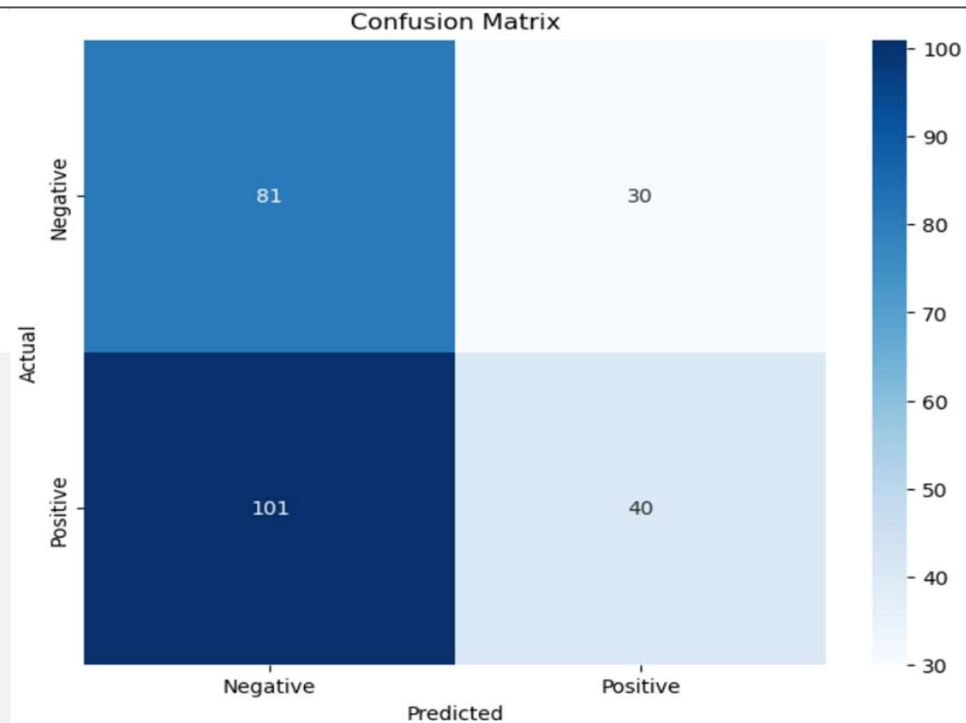
Classification Report:

	precision	recall	f1-score	support
0	0.45	0.73	0.55	111
1	0.57	0.28	0.38	141
accuracy			0.48	252
macro avg	0.51	0.51	0.47	252
weighted avg	0.52	0.48	0.46	252

MODEL COEFFICIENTS & CONFUSION MATRIX

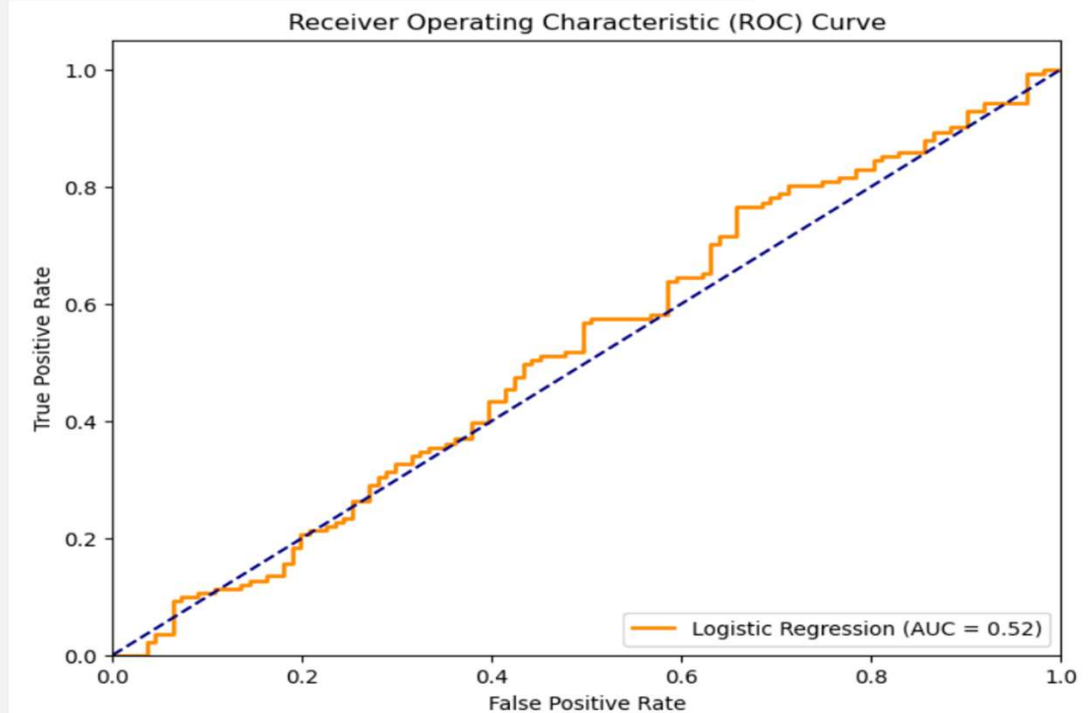
Model Coefficients:

	Coefficient
Lag1	-0.059062
Lag2	-0.042119
Lag3	0.008779
Lag4	0.000466
Lag5	-0.004212
Volume	-0.120205

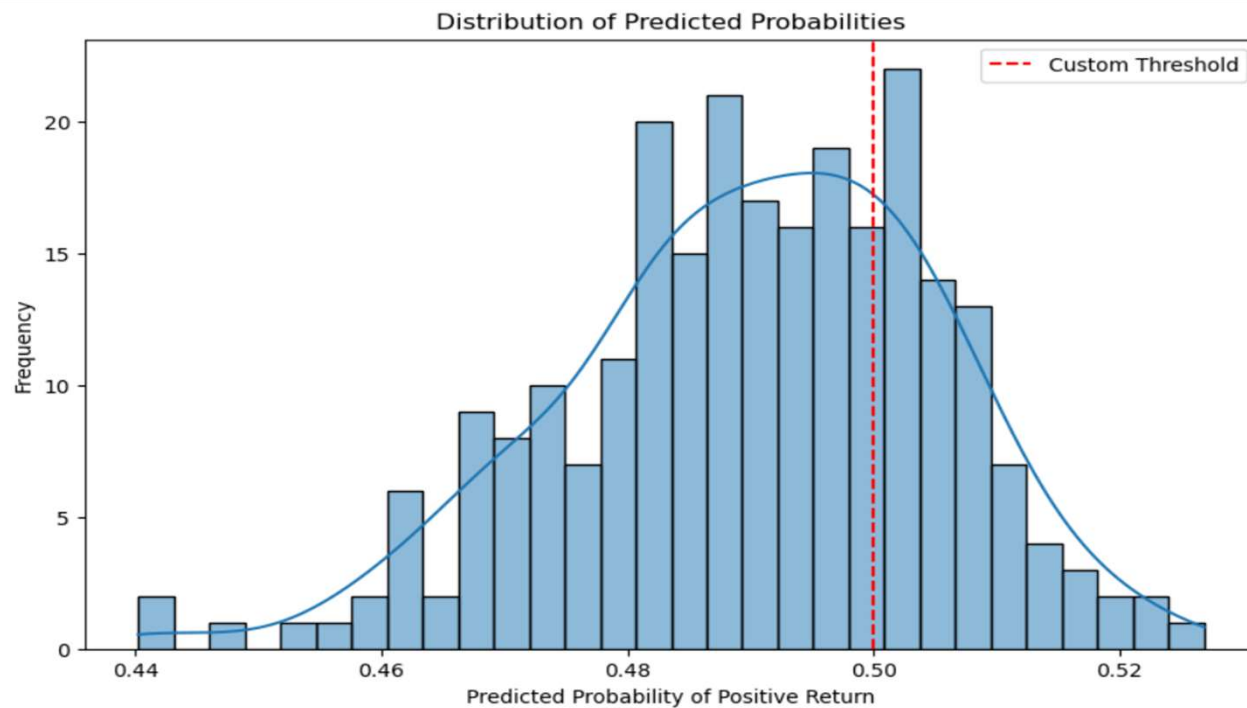


ROC

- Here we can see that the curve is very close to diagonal line ie. the model does not perform better than random guessing.
- Our AUC is 0.52 that indicates that the model has slight ability to differentiate between classes, but it is not very effective



VISUALIZATION OF PREDICTED PROBABILITIES



A FUNCTION TO CALCULATE THE TOTAL RETURN WHEN IT IS POSITIVE

```
def trading_strategy(data, initial_investment=1000, day_return=0):  
    cash = initial_investment # Start with the initial investment  
    for i, row in data.iterrows():  
        # If expected return (Today) is greater than the threshold  
        if row['Today'] > day_return:  
            # Invest all money, multiplying by the return percentage  
            cash *= (1 + row['Today'] / 100)  
    return cash  
final_cash_0_return = trading_strategy(Smarket, day_return=0)  
print(f"Final cash with 0% day return: ${final_cash_0_return:.2f}")  
Final cash with 0% day return: $174605.77
```

KEEPING RETURN PERCENTAGE Y CONSTANT

Q. At the beginning of any day, if you expect the percentage return to be greater than $y\%$, invest all of the money currently on hand, otherwise do not invest for the upcoming day.

```
def trading_strategy_varying_y(data, initial_investment=1000, y_value=0):  
    # Execute the trading strategy with a single threshold value  
    final_cash = trading_strategy(data, initial_investment, y_value)  
    return y_value, final_cash  
  
# Test with a specific threshold value for y:  
y_value = 0.5 # Test threshold of +0.5%  
y, final_cash = trading_strategy_varying_y(Smarket, initial_investment=1000, y_value=y_value)  
  
# Print the result for the single threshold  
print(f"Final cash with {y}% threshold: ${final_cash:.2f}")
```

Final cash with 0.5% threshold: \$89398.31

VARYING Y

```
def trading_strategy_varying_y(data, initial_investment=1000, y_values=[0]):
    results = []
    for y in y_values:
        final_cash = trading_strategy(data, initial_investment, y)
        results.append((y, final_cash))
    return results

# Now Let's try a range of thresholds for y:
y_values = [-2, -1, 0, 1, 2] # Test thresholds of -2%, -1%, 0%, +1%, +2%
results = trading_strategy_varying_y(test_data, initial_investment=1000, y_values=y_values)

# Print results for different thresholds
for y, final_cash in results:
    print(f"Final cash with {y}% threshold: ${final_cash:.2f}")

Final cash with -2% threshold: $1029.95
Final cash with -1% threshold: $1263.96
Final cash with 0% threshold: $1943.64
Final cash with 1% threshold: $1188.39
Final cash with 2% threshold: $1000.00
```

IMPROVING STRATEGY

#how would you improve the above strategies?

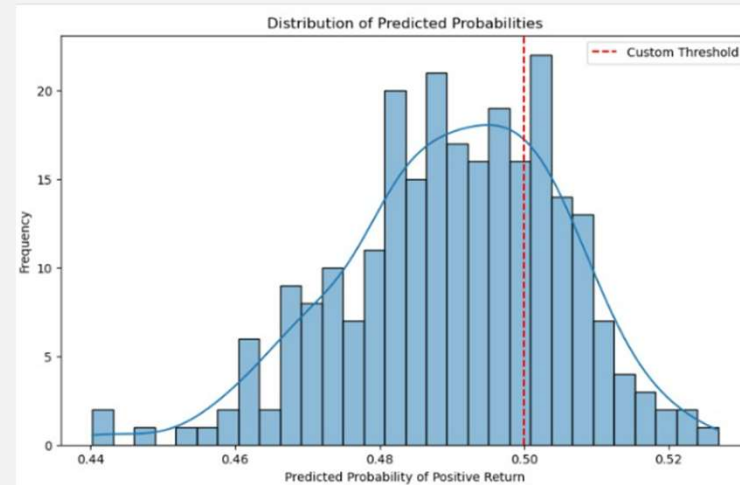
Adaptive Thresholds (Dynamic y) Volatility-Based Adjustments: Instead of using a static threshold, adjust y based on the market's volatility. For instance, in more volatile markets, a wider y range may be necessary, while in calm markets, a tighter y threshold would be appropriate. Volatility measures like the standard deviation or average true range (ATR) can guide this. **Moving Average or Trend Indicators:** Use moving averages or other trend-following indicators to dynamically adjust y . For example, if the market is in an uptrend (determined by the slope of the moving average), set a more aggressive y threshold, and if the market is flat or down, reduce the threshold.

IMPROVING THE STRATEGIES

- Add real market factors: macroeconomic data (GDP Growth, Interest Rate, Inflation)
- Use more features like volume or lagged returns to improve predictions.
- Experiment with different thresholds based on historical analysis or optimization techniques like grid search.

THRESHOLD SELECTION IN LOGISTIC REGRESSION

```
# Optional: Visualization of predicted probabilities
plt.figure(figsize=(10, 6))
sns.histplot(y_probs, bins=30, kde=True)
plt.axvline(x=custom_threshold, color='red', linestyle='--', label='Custom Threshold')
plt.xlabel('Predicted Probability of Positive Return')
plt.ylabel('Frequency')
plt.title('Distribution of Predicted Probabilities')
plt.legend()
plt.show()
```



answer : the threshold for prediction with your logistic regression model was done based using this graph

HOW DOES THRESHOLD SELECTION IN LOGISTIC REGRESSION DIFFER FROM Y IN TRADING?

The logistic regression threshold controls the probability required for the model to predict a positive return (e.g., a 50% probability threshold means the model must be at least 50% confident).

The y threshold in the trading strategy controls the predicted return percentage needed to invest (e.g., only invest if the return is expected to be greater than 1%).

In summary:

Logistic regression threshold: Based on probability (confidence in positive return). y threshold: Based on predicted return percentage (magnitude of return). They differ in what they measure: one is model confidence, the other is the size of the return.

THE BEST STRATEGY

Dollar-cost averaging combined with stop-losses and diversification offers a solid, low-risk strategy for uncertain conditions. This allows you to participate in potential gains while managing the downside effectively

1. Dollar-Cost Averaging (DCA)

Description: Invest a fixed amount of money at regular intervals, regardless of market conditions. For example, you invest a fixed percentage of your portfolio each day.

Rationale: This method averages out the purchase price over time. On bad days, you buy more at lower prices, and on good days, you buy less at higher prices.

Benefit: It smoothens out market volatility and reduces the emotional burden of trying to time the market.

Best for: Long-term investors who want to mitigate risk without needing to predict market movements.

2. Set a Stop-Loss Limit

Description: Place a stop-loss order that automatically sells part of your investment if the return drops below a certain percentage (e.g., -0.5%).

Rationale: You may not know in advance if the day will be bad, but you can limit your losses once you realize it's a bad day. You can also use a trailing stop-loss, which adjusts as your investment grows.

Benefit: Limits the downside during bad days while still allowing you to capture upside on good days.